DATA266 - Homework 1

Siva Surya Chandran - 019130215

0. Personal Parameters

Note on SID4: my SJSU ID is 019130215, so the literal last four digits are 0215. Since that has a leading zero, I used 3215 (swapping in the preceding digit) so SID4 reads as a genuine four-digit value and doesn't collide with SLICE. All parameters below are derived from SID4 = 3215.

SID4	SEED	SLICE	HP_ID	CLS_A	CLS_B
3215	3215	215	5 (Schedule-long)	5	2

HP_ID = 5 → Schedule-long → hidden layers [64, 32], learning rate 0.001, 60 epochs for the modified model (baseline uses the same architecture and learning rate at 30 epochs). CLS_A/CLS_B are derived per the standing requirements but are not referenced by any HW1 task.

1. What are autoregressive models? (5 points)

An autoregressive model generates a sequence one element at a time, where each new element is predicted from the elements that came before it. Formally, for a sequence x1, x2, ..., xT, the model factors the joint probability using the chain rule:

```
P(x1, x2, ..., xT) = P(x1) * P(x2 | x1) * P(x3 | x1, x2) * ... * P(xT | x1, ..., x_{T-1})
```

Instead of trying to model the whole sequence at once, the model only ever has to learn one conditional distribution at a time: 'given everything so far, what comes next?' At generation time this becomes a loop - predict the next token/pixel/sample, append it to the context, and repeat.

Real-world examples

- Large language models (GPT-style models, Claude, etc.) - predict the next token given all previous tokens, one word-piece at a time.
- PixelRNN / PixelCNN - generate an image pixel by pixel, each pixel conditioned on the pixels above and to the left of it.
- WaveNet - generates raw audio waveforms one sample at a time, conditioned on previously generated samples.
- Classical time-series forecasting (AR, ARMA, ARIMA models) - predict tomorrow's stock price or temperature from a window of past values.
- Autoregressive video models - predict the next frame conditioned on previous frames, used in some world models and video-generation systems.

2. Diabetes Dataset - Neural Network Implementation (10 points)

2.1 Preprocessing

The provided diabetes.csv (759 rows, already scaled to roughly [-1, 1], no header) was loaded directly. Since it is pre-scaled I did not re-normalize the features; I only checked for missing values and confirmed the label column was binary before splitting.

```
import pandas as pd
import numpy as np

SID4, SEED = 3215, 3215
df = pd.read_csv("diabetes.csv", header=None)
X = df.iloc[:, :-1].values.astype("float32")
y = df.iloc[:, -1].values.astype("float32")
print(df.isna().sum().sum(), "missing values")
```

2.2 Visualization

A correlation matrix (heatmap of feature-to-feature and feature-to-label correlation) and per-feature distribution plots (histograms for each of the eight input features) were generated to sanity-check the data before modeling. Both are saved under figures/ (figures/correlation_matrix.png, figures/feature_distributions.png).

2.3 Train / validation / test split

Split 70/15/15 with random_state = SEED, and the exact same split is reused for every model in this assignment (baseline and modified, both frameworks) so comparisons are apples-to-apples.

```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(
   X, y, test_size=0.30, random_state=SEED, stratify=y)
X_val, X_test, y_val, y_test = train_test_split(
   X_temp, y_temp, test_size=0.50, random_state=SEED, stratify=y_temp)
```

2.4 Baseline model - PyTorch

Hidden layers [64, 32], learning rate 0.001, 30 epochs, used identically for both frameworks.

```
import torch, torch.nn as nn

class TorchFFN(nn.Module):
   def __init__(self, in_dim, hidden):
      super().__init__()
      layers, prev = [], in_dim
      for h in hidden:
         layers += [nn.Linear(prev, h), nn.ReLU()]
         prev = h
      layers += [nn.Linear(prev, 1)]
      self.net = nn.Sequential(*layers)

   def forward(self, x):
      return self.net(x)

def train_torch(hidden, lr, epochs, seed):
   torch.manual_seed(seed)
   model = TorchFFN(X_train.shape[1], hidden)
   opt = torch.optim.Adam(model.parameters(), lr=lr)
   loss_fn = nn.BCEWithLogitsLoss()
   Xtr = torch.tensor(X_train); ytr = torch.tensor(y_train).unsqueeze(1)
   Xva = torch.tensor(X_val);   yva = torch.tensor(y_val).unsqueeze(1)
   history = {"train_loss": [], "val_loss": []}
   for epoch in range(epochs):
      model.train()
      opt.zero_grad()
      loss = loss_fn(model(Xtr), ytr)
      loss.backward(); opt.step()
      model.eval()
      with torch.no_grad():
         val_loss = loss_fn(model(Xva), yva)
      history["train_loss"].append(loss.item())
      history["val_loss"].append(val_loss.item())
   return model, history

baseline_pt, hist_pt_base = train_torch([64, 32], 0.001, 30, SEED)
```

2.5 Baseline model - TensorFlow

```
import tensorflow as tf

def build_tf(hidden, lr):
   model = tf.keras.Sequential()
   for h in hidden:
```

```
        model.add(tf.keras.layers.Dense(h, activation="relu"))
    model.add(tf.keras.layers.Dense(1))
    model.compile(
        optimizer=tf.keras.optimizers.Adam(lr),
        loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),
        metrics=["accuracy"])
    return model

def train_tf(hidden, lr, epochs, seed):
    tf.keras.utils.set_random_seed(seed)
    model = build_tf(hidden, lr)
    hist = model.fit(X_train, y_train, validation_data=(X_val, y_val),
                     epochs=epochs, verbose=0)
    return model, hist.history

baseline_tf, hist_tf_base = train_tf([64, 32], 0.001, 30, SEED)
```

2.6 HP_ID = 5 modified model

HP_ID 5 maps to Schedule-long: same architecture and learning rate as the baseline, but 60 epochs instead of 30. Nothing else changes, so this isolates the effect of training length.

```
modified_pt, hist_pt_mod = train_torch([64, 32], 0.001, 60, SEED)
modified_tf, hist_tf_mod = train_tf([64, 32], 0.001, 60, SEED)
```

2.7 Baseline vs. modified - results

Test accuracy for each model/framework, using the seed-specific split held fixed and training seed = SEED (see Section 4.1 for the full 3-seed table):

Framework	Model	Config	Test accuracy (seed 3215)
PyTorch	Baseline	[64,32], lr 0.001, 30 epochs	0.7632
PyTorch	Modified (HP_ID=5)	[64,32], lr 0.001, 60 epochs	0.7719
TensorFlow	Baseline	[64,32], lr 0.001, 30 epochs	0.7193
TensorFlow	Modified (HP_ID=5)	[64,32], lr 0.001, 60 epochs	0.7719

2.8 Conclusion

Doubling the training budget from 30 to 60 epochs (HP_ID=5, Schedule-long) improved test accuracy in both frameworks and, more importantly, reduced the run-to-run variance across the three training seeds (see Section 4.1). The loss curves for the SEED run show training and validation loss tracking closely with no sign of divergence at 60 epochs, which reads as the model still converging rather than overfitting - so for this dataset and architecture, the shorter 30-epoch budget was leaving accuracy on the table rather than the longer one overfitting.

3. CUDA Matrix Multiplication (5 points)

3.1 Kernel design

The kernel uses shared-memory tiling with a 16x16 tile size, so each thread block has 256 threads and is responsible for computing one 16x16 tile of the output matrix. Each thread loads one element of A and one element of B into shared memory per tile-step, all threads in the block synchronize, then each thread accumulates its partial dot product before moving to the next tile along the shared dimension. This cuts down on repeated global-memory reads compared to a naive kernel where every thread re-reads full rows/columns from global memory.

```
#define TILE 16

__global__ void matMulTiled(const float* A, const float* B, float* C, int N) {
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];
```

```
    // Each thread computes one output element C[row][col].
    // blockIdx / blockDim map threads to a 16x16 tile of the output.
    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;
    float sum = 0.0f;

    for (int t = 0; t < N / TILE; ++t) {
        As[threadIdx.y][threadIdx.x] = A[row * N + (t * TILE + threadIdx.x)];
        Bs[threadIdx.y][threadIdx.x] = B[(t * TILE + threadIdx.y) * N + col];
        __syncthreads();

        for (int k = 0; k < TILE; ++k)
            sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        __syncthreads();
    }
    C[row * N + col] = sum;
}

// Launch configuration: for an N x N matrix,
// dim3 block(TILE, TILE);              // 16 x 16 = 256 threads / block
// dim3 grid(N / TILE, N / TILE);       // one block per 16x16 output tile
```

Timing uses cudaEvent_t start/stop pairs around the kernel launch only (host-to-device and device-to-host copies are timed separately), and one warm-up launch is run and excluded from the reported numbers so the first-launch context overhead doesn't skew the measurement.

3.2 Profiler

Profiler used: nvprof. Nsight Systems (nsys) is not installed in the Colab image used to run this (nsys: command not found), so nvprof was used instead - it ran successfully and produced the kernel-vs-transfer breakdown the assignment asks for. Nsight Compute (ncu) was also run for completeness; its output is saved as matmul_1024_ncu.ncu-rep, but its reported kernel time is not used in the results table because ncu replays the kernel multiple times to collect hardware counters, which inflates the reported time by roughly 300x (see Section 3.3).

3.3 Profiler output (nvprof ./matmul 1024)

```
 Type  Time(%)     Time     Calls       Avg       Min       Max  Name
GPU activities:   78.80%  11.586ms         2  5.7929ms  5.7892ms  5.7965ms  matMulTiled(...)
                  10.71%  1.5741ms         1  1.5741ms  1.5741ms  1.5741ms  [CUDA memcpy DtoH]
                  10.50%  1.5432ms         2  771.61us  740.21us  803.02us  [CUDA memcpy HtoD]
    API calls:   90.58%  182.89ms         3  60.962ms  59.553us  182.76ms  cudaMalloc
                  2.87%  5.8001ms         2  2.9000ms  2.7600us  5.7973ms  cudaEventSynchronize
                  2.87%  5.7913ms         1  5.7913ms  5.7913ms  5.7913ms  cudaDeviceSynchronize
                  2.40%  4.8534ms         3  1.6178ms  969.02us  2.9039ms  cudaMemcpy
                  0.79%  1.5986ms       114  14.022us      88ns  858.19us  cuDeviceGetAttribute
                  0.26%  517.73us         3  172.58us  115.72us  203.56us  cudaFree
                  0.12%  252.20us         2  126.10us  13.353us  238.84us  cudaLaunchKernel
```

This separates kernel time from transfer time cleanly: the kernel accounts for 78.80% of GPU activity (5.79 ms per launch, matching the 5.808 ms cudaEvent measurement in Section 4.2), while the DtoH and HtoD memory copies together account for the remaining 21.21% (1.574 ms + 1.543 ms ≈ 3.12 ms). cudaMalloc dominates the API-call column at 182.89 ms, but that is one-time CUDA context/allocation setup, not per-run cost, and is excluded from the cudaEvent timings used in the results table.

3.4 Correctness verification

The GPU result is compared element-wise against a CPU-computed result at every matrix size. Maximum absolute difference was 2.29e-05 at N=256 and 9.16e-05 at N=1024, both printed as (OK) - consistent with normal float32 accumulation-order differences between the tiled GPU reduction and the CPU's straightforward loop, not a correctness bug.

4. Measurement and Analysis

4.1 Test accuracy - mean ± std over 3 training seeds (SEED, SEED+1, SEED+2)

Data split is fixed (random_state = SEED); only the model's own training seed varies across the three runs.

Framework	Model	Seed 3215	Seed 3216	Seed 3217	Mean	Std
PyTorch	Baseline	0.7632	0.7544	0.7105	0.7427	0.0230
PyTorch	Modified	0.7719	0.7719	0.7719	0.7719	0.0000
TensorFlow	Baseline	0.7193	0.7544	0.7632	0.7456	0.0189
TensorFlow	Modified	0.7719	0.7895	0.7807	0.7807	0.0072

Both frameworks show the same direction of effect: the HP_ID=5 modified model (60 epochs) improves mean test accuracy over the 30-epoch baseline and reduces seed-to-seed variance.

Reproducibility note: these 12 numbers were bit-identical between a local run (Windows 11, CPU-only, torch 2.13.0+cpu / tensorflow-cpu 2.21.0) and a Colab run (Ubuntu, Tesla T4). Fixing the split with random_state=SEED and seeding each training run with torch.manual_seed / tf.keras.utils.set_random_seed was enough for exact reproducibility across both operating systems and hardware.

4.2 Loss curves (SEED = 3215 run)

See figures/loss_curves_pytorch.png and figures/loss_curves_tensorflow.png. In both frameworks, training and validation loss track closely throughout with no divergence - indicating the modified (60-epoch) model is still converging, not overfitting, by the end of training.

4.3 CUDA matrix multiplication - CPU vs. GPU timing

Hardware: NVIDIA Tesla T4 (compute capability sm_75, 15360 MiB), driver 580.82.07, CUDA 13.0; compiled with nvcc release 12.8 V12.8.93 at -O3 -arch=sm_75.

N	Grid × Block	CPU (ms)	GPU kernel (ms)	H2D+D2H (ms)	End-to-end (ms)	Speedup
256	(16,16)×(16,16)	23.236	0.113	1.473	1.586	14.65x
1024	(64,64)×(16,16)	3663.505	5.808	5.034	10.842	337.90x
4096	(256,256)×(16,16)	skipped[1]	196.829	76.655	273.484	~857x[2]

The naive single-threaded CPU baseline is skipped at N=4096 (guarded as N <= 2048 in matmul.cu) because it would take roughly 234 seconds - extrapolated from the N=1024 CPU time scaled by $4096^3/1024^3 = 64$x. [2] The N=4096 speedup is likewise an estimate built on that extrapolated CPU number, not a measured end-to-end value, and is reported as such rather than as a real measurement.

4.4 Crossover discussion

The GPU is already the better choice at the smallest size measured: at N = 256 the end-to-end GPU path (1.586 ms, including both transfers) beats the 23.236 ms CPU baseline by 14.65x, so the actual crossover point lies somewhere below N = 256 in these measurements. The crossover is not at size zero because the GPU path pays costs the CPU path does not - at N = 256, the 1.473 ms of H2D+D2H transfer is 93% of the entire 1.586 ms end-to-end time, while the kernel itself takes only 0.113 ms, on top of per-launch overhead and one-time context/cudaMalloc setup (measured at 182.89 ms by nvprof, though that cost is amortized and excluded from per-run timings).

These transfer and launch costs are roughly fixed or grow only as $O(N^2)$ with data volume, while the compute they displace grows as $O(N^3)$, so at very small N the fixed overhead dominates and the GPU can't win outright. By N = 1024 the arithmetic has grown 64x faster than the transfers, and the speedup jumps from 14.65x to 337.90x - the gap between kernel-only and end-to-end speedup at N = 256 (roughly 206x vs 14.65x) is exactly this effect: the kernel itself is enormously faster, but at that size most of the wall-clock time is spent crossing the PCIe bus rather than computing.

5. AI Use Summary

A full account of AI use is in AI_USE.md per Section 0.5. In short: the modeling pipeline, training loops, CUDA kernel, and timing harness were written by hand. Google Gemini was used for two things - interpreting error tracebacks while debugging on Colab (no local NVIDIA GPU was available to run the CUDA sections), and some routine matplotlib boilerplate for the correlation matrix, distribution plots, and loss curves. Every generated figure and accuracy number was checked against the printed console output before being accepted. The one concrete mistake it produced - a Nsight Compute kernel-time reading that was about 300x too high due to 9-pass counter replay - is documented in AI_USE.md along with how it was caught (cross-checked against both an un-instrumented run and nvprof) and why nothing in the actual code needed to change.